

minimum_node_cut

`minimum_node_cut` (`G`, `s=None`, `t=None`, `flow_func=None`) [\[source\]](#)

Returns a set of nodes of minimum cardinality that disconnects `G`.

If source and target nodes are provided, this function returns the set of nodes of minimum cardinality that, if removed, would destroy all paths among source and target in `G`. If not, it returns a set of nodes of minimum cardinality that disconnects `G`.

Parameters:

- `G` (*NetworkX graph*) –
- `s` (*node*) – Source node. Optional. Default value: `None`.
- `t` (*node*) – Target node. Optional. Default value: `None`.
- `flow_func` (*function*) – A function for computing the maximum flow among a pair of nodes. The function has to accept at least three parameters: a `Digraph`, a source node, and a target node. And return a residual network that follows NetworkX conventions (see `maximum_flow()` for details). If `flow_func` is `None`, the default maximum flow function (`edmonds_karp()`) is used. See below for details. The choice of the default function may change from version to version and should not be relied on. Default value: `None`.

Returns: `cutset` – Set of nodes that, if removed, would disconnect `G`. If source and target nodes are provided, the set contains the nodes that if removed, would destroy all paths between source and target.

Return type: `set`

Examples

```
>>> # Platonic icosahedral graph has node connectivity 5
>>> G = nx.icosahedral_graph()
>>> node_cut = nx.minimum_node_cut(G)
>>> len(node_cut)
5
```

You can use alternative flow algorithms for the underlying maximum flow computation. In dense networks the algorithm `shortest_augmenting_path()` will usually perform better than the default `edmonds_karp()`, which is faster for sparse networks with highly skewed degree distributions. Alternative flow functions have to be explicitly imported from the flow package.

```
>>> from networkx.algorithms.flow import shortest_augmenting_path
>>> node_cut == nx.minimum_node_cut(G, flow_func=shortest_augmenting_path)
True
```

If you specify a pair of nodes (source and target) as parameters, this function returns a local st node cut.

```
>>> len(nx.minimum_node_cut(G, 3, 7))
5
```

If you need to perform several local st cuts among different pairs of nodes on the same graph, it is recommended that you reuse the data structures used in the maximum flow computations. See `minimum_st_node_cut()` for details.

Notes

This is a flow based implementation of minimum node cut. The algorithm is based in solving a number of maximum flow computations to determine the capacity of the minimum cut on an auxiliary directed network that corresponds to the minimum node cut of G. It handles both directed and undirected graphs. This implementation is based on algorithm 11 in [\[1\]](#).

! See also

`minimum_st_node_cut()`, `minimum_cut()`, `minimum_edge_cut()`, `stoer_wagner()`,
`node_connectivity()`, `edge_connectivity()`, `maximum_flow()`, `edmonds_karp()`,
`preflow_push()`, `shortest_augmenting_path()`

References

- [1] Abdol-Hossein Esfahanian. Connectivity Algorithms.
http://www.cse.msu.edu/~cse835/Papers/Graph_connectivity_revised.pdf